

1) Listes linéaires simplement chaînées

EXERCICE 1

On souhaite calculer le PGCD de n entiers donnés. On suppose que les entiers sont tous distincts et supérieurs à 1. Les n entiers sont stockés dans un tableau `tab` d'entiers.

Pour calculer le PGCD des n entiers, on décompose chaque entier en facteurs premiers. La décomposition d'un entier en facteurs premiers sera représentée par une liste chaînée dont la structure est définie comme suit :

```
typedef struct{
    int fact ;    // indique le facteur premier
    int nbOcc ;  // nombre d'occurrences du facteur premier
}FactPremier;

typedef struct cellule{
    FactPremier donnee;
    struct cellule *suivant ;
}*Liste;
```

Exemple 1 : la décomposition de l'entier $40=2^3 \cdot 5$ en facteurs premiers est représentée par la liste chaînée suivante :



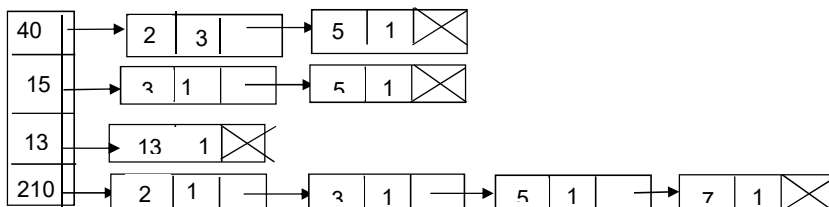
Par suite, le tableau d'entiers « `tab` » sera transformé en un tableau de listes chaînées « `tabListe` ».

Exemple 2 :

Tab=

40	15	13	210
----	----	----	-----

tabListe



- 1) Ecrire en langage C, la structure de données représentant un tableau de listes chaînées selon l'exemple 2.
- 2) Ecrire en langage algorithmique (ou en C), la décomposition d'un entier donné en facteurs premiers. Les facteurs premiers seront représentés par une liste chaînée (comme ci-décrit dans l'exemple 1).
- 3) En utilisant la question 2, écrire en langage algorithmique (ou en C), la construction du tableau de listes (`TabListe`) à partir d'un tableau d'entiers (comme en exemple 2).
- 4) Ecrire en langage algorithmique (ou en C), l'affichage des facteurs premiers de chaque entier du tableau de listes.
- 5) Ecrire en langage algorithmique (ou en C), l'intersection de deux listes chaînées représentant les facteurs premiers. Que représente l'intersection de deux listes ?
- 6) En utilisant la question 5, déterminer le PGCD de n entiers donnés.
- 7) Proposer un programme principal avec un **menu** pour tester les fonctions définies ci-dessus.

EXERCICE 2

Réécrire la structure de données ainsi que les algorithmes précédents en supposant que les nombres sont stockés dans une liste linéaire simplement chaînée d'une part et d'autre part la décomposition des nombres est stocké également dans une liste de listes des facteurs premiers.

EXERCICE 3

Réécrire les structures de données ainsi que les algorithmes précédents en supposant que les listes chaînées sont définies par deux adresses : la première adresse pour indiquer la tête de la liste et la seconde adresse pour indiquer la queue de la liste.

EXTRAIT PARTIEL NOVEMBRE 2023**EXERCICE 3 (10.5PTS) : LISTES CHAÎNÉES**

On souhaite gérer une **liste chaînée triée** par ordre croissant où les **données de la liste n'apparaissent qu'une seule fois**.

La structure de la liste chaînée est définie comme suit :

```
typedef int TElement ;
typedef struct cellule{
    TElement donnee;
    struct cellule * suivant;
}*Liste ;
```

- 8) Ecrire en langage algorithmique ou en C, un algorithme qui recherche un élément donné dans une liste chaînée triée par ordre croissant donné. Cet algorithme retourne l'adresse de la cellule contenant l'élément s'il existe ou NULL si non.
- 9) **Ecrire en C**, un algorithme qui insère un élément donné dans une liste chaînée triée donnée.
- 10) **Ecrire en C**, un algorithme qui supprime un élément donné dans une liste chaînée triée donnée.

Par la suite, on souhaite gérer un tableau de listes chaînées qu'on nommé **table**. Les listes sont triées par ordre croissant. Pour cela on définit la structure en C comme suit :

```
typedef struct{
    int taille;    // la dimension du tableau
    Liste *tab;    // tableau de listes chaînées
}Table;
```

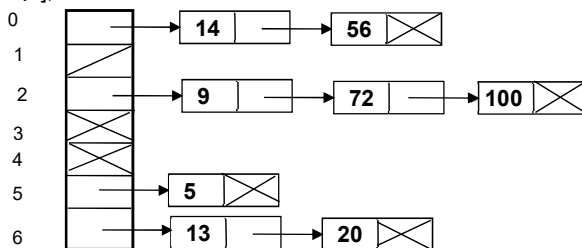
On suppose que la taille de la table est connue et les données seront insérées comme suit :

Pour insérer un élément « elt » dans la table on détermine d'une part sa position « pos » dans la table, et d'autre part son insertion dans la liste chaînée tab[pos]. **L'élément est inséré s'il n'existe pas dans la table (c'est-à-dire s'il n'appartient pas à la liste chaînée tab[pos]).**

La position est définie par la formule suivante : $pos = elt \text{ modulo } \text{taille}$.

Exemple : taille=7, les éléments 72, 14, 100, 13, 14, 56, 9, 5, 13, et 20 sont insérés dans la table comme ci-dessous :

72 modulo 7 = 2, 14 modulo 7 = 0, 100 modulo 7 = 2, 13 modulo 7 = 6,
56 modulo 7 = 0, 9 modulo 7 = 2, 5 modulo 7 = 5, 20 modulo 7 = 6,
72, 9, 100 sont insérés dans la liste chaînée tab[2], 14, 56 sont insérés dans la liste chaînée tab[0], etc



Remarque : Les données dans la table n'apparaissent qu'une seule fois ; par exemple 14 et 13.

- 11) **Ecrire en C**, l'algorithme d'allocation mémoire d'une table en fonction d'une taille donnée.
- 12) **Ecrire en C**, l'algorithme de libération mémoire d'une table donnée.
- 13) Ecrire en langage algorithmique ou en C, l'algorithme d'initialisation d'une table donné.
- 14) **Ecrire en C**, l'algorithme d'insertion d'un élément donné dans une table donnée.

- 15) Ecrire en langage algorithmique ou en C, un algorithme **itératif** qui remplit une table à partir d'un tableau donné composé de n éléments.
- 16) Ecrire en langage algorithmique ou en C, un algorithme **récuratif** qui remplit une table à partir d'un tableau donné composé de n éléments.
- 17) Ecrire en langage algorithmique ou en C, un algorithme qui retourne le nombre d'éléments présents dans la table.
- 18) Ecrire en langage algorithmique ou en C ; un **algorithme itératif** qui détermine l'élément de la table qui possède le plus d'éléments.
- 19) **Ecrire en C ; un algorithme récuratif** qui détermine l'élément de la table qui possède le plus d'éléments.
- 20) Ecrire en langage algorithmique ou en C, un algorithme qui affiche une table donnée.
- 21) **Ecrire en C**, l'algorithme de suppression d'un élément donné dans une table donnée.
- 22) **Compléter en C le programme suivant :**

```
void main(void){
    TElement tab[DIM]={14, 72, 99, 7, 0, 13, 14, 5, 6, 2, 12, 15, 19,
        33, 47, 13, 0}, n=17;
    TElement elt;
    Table t;

    // Allocation mémoire de la table
    ...
    // Initialisation de la table
    ...
    // Construction de la table à partir d'un tableau
    // appel de la version itérative
    ...
    // appel de la version récurative
    ...
    // Affichage du contenu de la table
    ...
    // Affichage du nombre d'éléments présent dans la table
    ...
    // Affichage de la plus grande liste de la table
    // appel de la version itérative
    ...
    // appel de la version récurative
    ...

    // suppression d'un élément de la table
    elt = 19;
    ...
    //Libération mémoire de la table
    ...
}
```